



IIT Kharagpur



IIT Madras



IIT Goa



IIT PALAKKAD

IIT Palakkad

End to End Accelerated Data Learning

Optimizing Deep Learning Training: Automatic Mixed Precision

Dr. Tosin Adesuyi
NVIDIA



National
Supercomputing
Mission



Centre for
Development of
Advanced Computing



Deep Learning Training Optimization

1/3

- **Optimizing deep learning training** is refer to as using optimizer algorithms or methods to tune a deep neural network to minimize losses and achieve desire accuracy.
- Tuning could be applied to:
 - Learning rate, Weights, etc
- An example of Basic Optimization Algorithm is the **Gradient descent**

Source: <https://towardsdatascience.com/optimizers-for-training-neural-network-59450d71caf6>

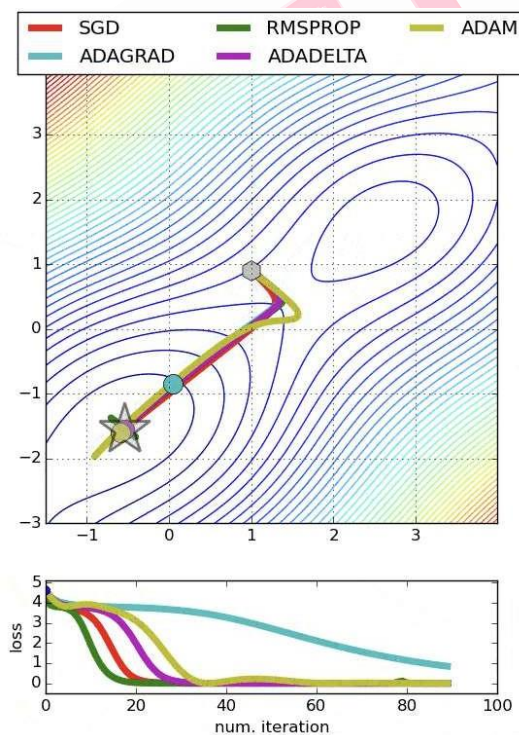


Deep Learning Training Optimization

2/3

- **Gradient descent**

- ✓ Stochastic Gradient Descent
- ✓ Mini-Batch Gradient Descent
- ✓ Momentum(reduce high variance SGD)



Source: <https://towardsdatascience.com/optimizers-for-training-neural-network-59450d71caf6>



Deep Learning Training Optimization

3/3

- **Advanced optimization of deep learning training phase**

- ✓ Memory consumption to accommodate large DNN
- ✓ Memory bandwidth to accommodate data transfer operation
- ✓ Tensor core computation for speedup

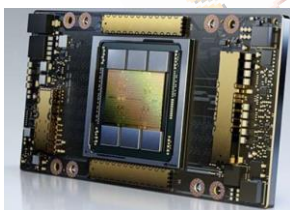
- **Examples include:**

- ✓ Acceleration with GPU
- ✓ **Mixed Precision**
- ✓ XLA(Accelerated Linear Algebra)
- ✓ Transfer Learning



Mixed Precision

- **Mixed Precision** is the combine use of different numerical format(single and half precision computation) in the training of a deep neural network.
- ✓ single precision: **FP32 (float32)**
- ✓ half precision: **FP16 (float16)**
- Mixed Precision is possible on the flowing GPU architectures:



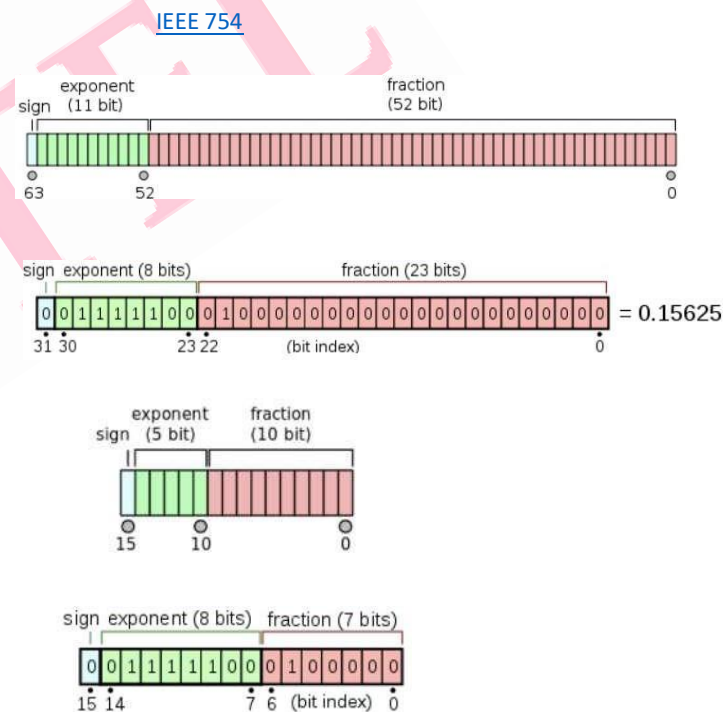
Ampere



Volta



Turing



<https://moocaholic.medium.com/fp64-fp32-fp16-bfloat16-tf32-and-other-members-of-the-zoo-a1ca7897d407>



Benefit of Mixed PrecisionTM

- **Speeds up math-intensive operations using Tensor Cores** (matrix-mult and convolution ops).
- **Require less memory bandwidth**, thus data transfer operations are speedup.
- **Require less memory**, thus the training and deployment of larger neural networks are possible.
- Volta and Turing architectures:
 - ✓ 3x overall speedup is achievable



Mixed Precision Training

- Training with mixed precision speedup computations by performing ops in half precision format.
- Minimal information are stored in single-precision to retain as much information in critical parts of the network.
- **Training steps**
 - Porting the model to use the FP16 data type where appropriate
 - Adding loss scaling to preserve small gradient values.



FP16 Porting in Model

- **Tensor core enabled framework**

- Choose **FP16** format for tensor, convolution, fully-connected layers,
- Keep all hyper-parameters of **FP32** training session.

- **Issue with the above method**

- ✓ Some network gradient becomes very tiny or lead to zero **0** (dynamic range too narrow, hence underflow)
- ✓ Required gradient value to be shifted into **FP16** representable range to match accuracy of **FP32** training session.

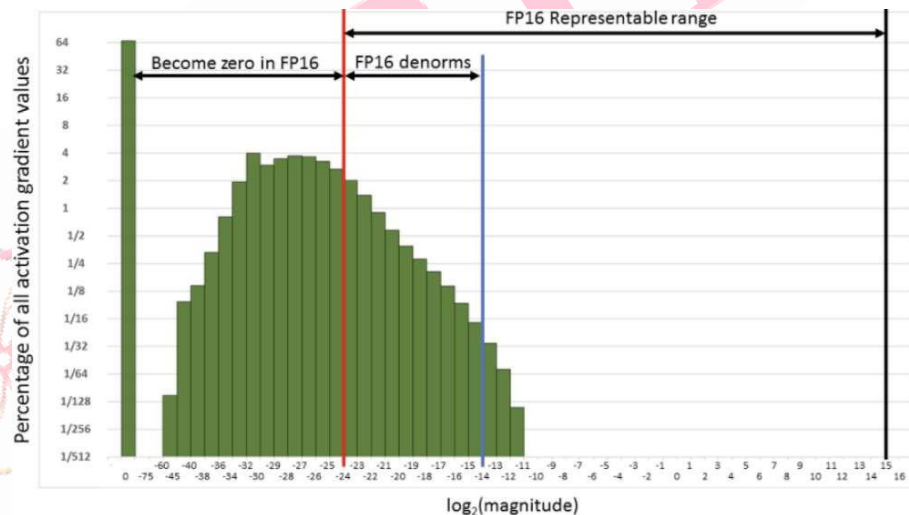
- **Solution**

- ✓ Scaling is required



FP16 Porting in Model

- FP16 dynamic range is sufficient for training, however gradients may require scaling to move them into the range to keep them from becoming zeros in FP16.
- Overflow should be avoided



Histogram of activation gradient magnitudes throughout FP32 training of Multibox SSD network. Both x- and y-axes are logarithmic.

Source: <https://docs.nvidia.com/deeplearning/performance/mixed-precision-training/index.html>



Loss Scaling

- The purpose of loss scaling is to preserve small gradient magnitudes.
- **Process**
 - A single multiplication by **scaling** the loss values computed in the **forward pass** prior to starting **backpropagation**
 - Weight gradients **unscaled** before weight update, to maintain the magnitude of updates the same as in **FP32** training
 - **FP32** copy of weights requires update and large reductions should be left in **FP32**

Training procedure

1. Maintain a primary copy of weights in **FP32**
2. For each iteration:
 - i. Make an **FP16** copy of the weights
 - ii. Forward propagation (**FP16** weights and activations)
 - iii. Multiply the resulting loss with the scaling factor **S**
 - iv. Backward propagation (**FP16** weights, activations, and their gradients)
 - v. Multiply the weight gradient with **1/S**
 - vi. Complete the weight update (including gradient clipping)

```
loss_scale = 1024
loss = model(inputs)
loss *= loss_scale

# Assume `grads` are float32. You do not want to divide float16 gradients.
grads = compute_gradient(loss, model.trainable_variables)
grads /= loss_scale
```



Choosing A Scale Factor

- **Process**

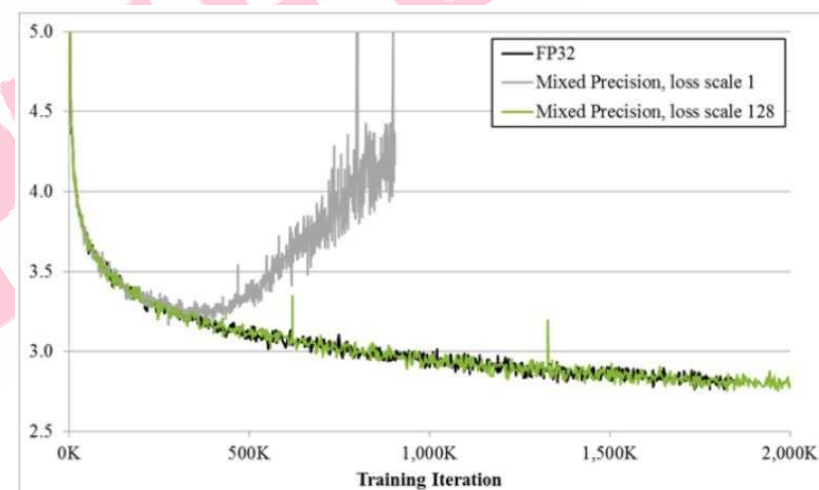
- Pick a constant scaling factor (if gradient statistics are available).
- some trial and error may be required because **scaling factor** can depend on the **network, framework, minibatch size**, etc.
- Choose a value so that its product with the maximum absolute gradient value is below 65,504 (the maximum value representable in FP16)

- **Avoiding Overflow**



Training Procedure in Choosing a Scale Factor

1. Maintain a primary copy of weights in **FP32**.
2. Initialize **S** to a large value.
3. For each iteration:
 - i. Make an **FP16** copy of the weights.
 - ii. Forward propagation (**FP16** weights and activations).
 - iii. Multiply the resulting loss with the scaling factor **S**.
 - iv. Backward propagation (**FP16** weights, activations, and their gradients).
 - v. If there is an **Inf** or **NaN** in weight gradients:
 - Reduce **S**.
 - Skip the weight update and move to the next iteration.
 - vi. Multiply the weight gradient with **1/S**.
 - vii. Complete the weight update (including gradient clipping, etc.).
 - viii. If there hasn't been an **Inf** or **NaN** in the last **N** iterations, increase **S**.



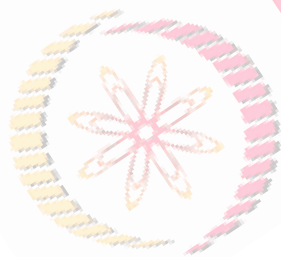
Training curves for the bigLSTM English language model shows the benefits of the mixed-precision training techniques. The Y-axis is training loss

Source: <https://docs.nvidia.com/deeplearning/performance/mixed-precision-training/index.html>

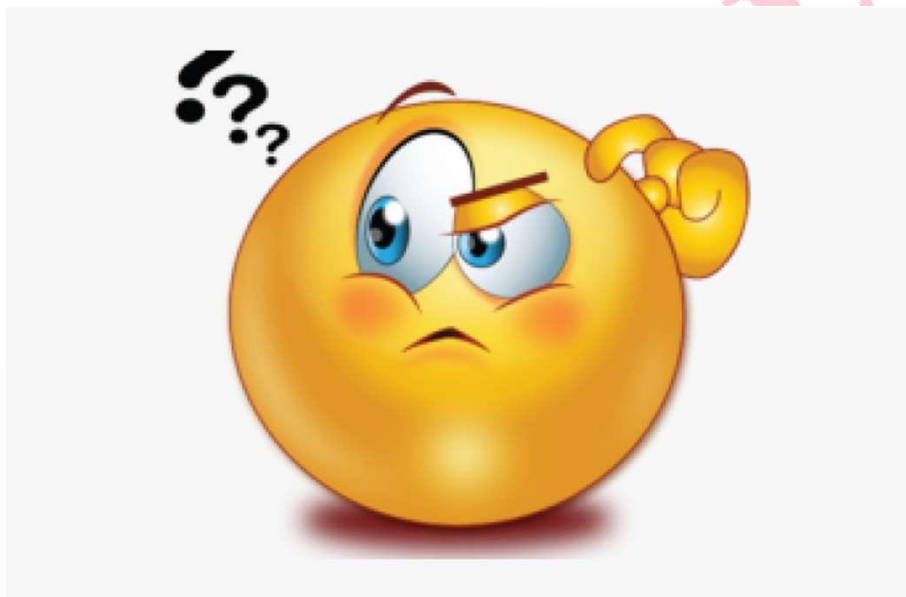


Summary: Mixed Precision Training

- Choose **FP16** format tensor core
- **Forward pass** of the model
- **Scale the loss** and backpropagate the scaled gradients
- **Un-scale** the gradients and optimizer performs the weight update



Is there a simpler way?

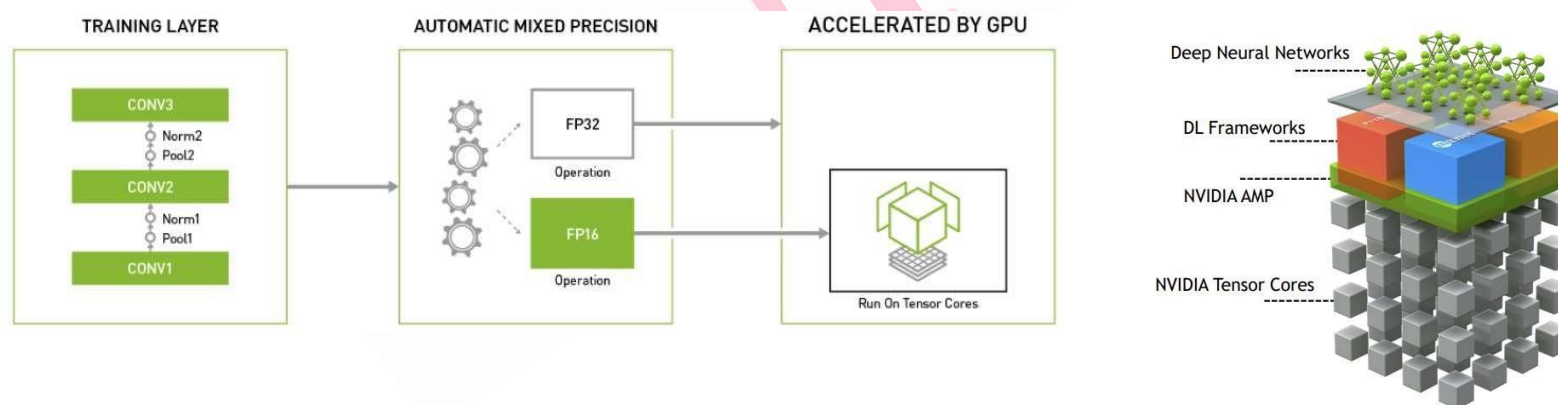




Automatic Mixed Precision (AMP)

Automatic Mixed Precision (AMP) makes mixed precision training with **FP16** easy in frameworks.

- AMP automates process of training in mixed precision
- Converts matrix multiplies/convolutions to 16-bits for Tensor Core acceleration



Source: <https://developer.nvidia.com/automatic-mixed-precision>



AMP Use

- **Frameworks**

- ✓ Through Scripting



PYTORCH



- **Containers**

- ✓ Tensorflow (19.07 or upstream TensorFlow 1.14 or later): on NVIDIA NGC
- ✓ Pytorch (22.02): Apex repository on GitHub & on NVIDIA NGC.
- ✓ Mxnet (19.04 or later): on NVIDIA NGC



DEMO





AMP: Tensorflow

set the environment variable inside a TensorFlow Python script

```
os.environ['TF_ENABLE_AUTO_MIXED_PRECISION'] = '1'
```

Graph-based:

```
opt = tf.train.AdamOptimizer()
```

```
opt = tf.train.experimental.enable_mixed_precision_graph_rewrite(opt)
```

```
train_op = opt.minimize(loss)
```

Keras-based:

```
opt = tf.keras.optimizers.Adam()
```

```
opt = tf.train.experimental.enable_mixed_precision_graph_rewrite(opt)
```

```
model.compile(loss=loss, optimizer=opt)
```

```
model.fit(...)
```

Source: <https://docs.nvidia.com/deeplearning/performance/mixed-precision-training/index.html>



AMP: Pytorch

```
import torch
# Creates once at the beginning of training
scaler = torch.cuda.amp.GradScaler()

for data, label in data_iter:
    optimizer.zero_grad()
    # Casts operations to mixed precision
    with torch.cuda.amp.autocast():
        loss = model(data)

    # Scales the loss, and calls backward()
    # to create scaled gradients
    scaler.scale(loss).backward()

    # Unscales gradients and calls
    # or skips optimizer.step()
    scaler.step(optimizer)

    # Updates the scale for next iteration
    scaler.update()
```

Source: <https://pytorch.org/blog/accelerating-training-on-nvidia-gpus-with-pytorch-automatic-mixed-precision/>

Thank You



NPTELTM